RESEARCH-ARTICLE

Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study

YUAN SUI, National University of Singapore, Singapore City, Singapore

MENGYU ZHOU, Microsoft Corporation, Redmond, WA, United States

MINGJIE ZHOU, The University of Hong Kong, Hong Kong, Hong Kong

SHI HAN, Microsoft Corporation, Redmond, WA, United States

DONGMEI ZHANG, Microsoft Corporation, Redmond, WA, United States

Open Access Support provided by:

Microsoft Corporation

National University of Singapore

The University of Hong Kong

Published: 04 March 2024

Citation in BibTeX format

WSDM '24: The 17th ACM International Conference on Web Search and Data Mining
March 4 - 8, 2024
Merida, Mexico

Conference Sponsors:
SIGKDD
SIGMOD
SIGIR
SIGWEB

Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study

Yuan Sui*
yuan.sui@u.nus.edu
National University of Singapore
Singapore

Mengyu Zhou†
mezho@microsoft.com
Microsoft
Beijing, China

Mingjie Zhou*
mjzhou@connect.hku.hk
The University of Hong Kong
Hong Kong, China

Shi Han
shihan@microsoft.com
Microsoft
Beijing, China

Dongmei Zhang
dongmeiz@microsoft.com
Microsoft
Beijing, China

ABSTRACT

Large language models (LLMs) are becoming attractive as few-shot reasoners to solve Natural Language (NL)-related tasks. However, there is still much to learn about how well LLMs understand structured data, such as tables. Although tables can be used as input to LLMs with serialization, there is a lack of comprehensive studies that examine whether LLMs can truly comprehend such data. In this paper, we try to understand this by designing a benchmark to evaluate the structural understanding capabilities (SUC) of LLMs. The benchmark we create includes seven tasks, each with its own unique challenges, *e.g.*, cell lookup, row retrieval, and size detection. We perform a series of evaluations on GPT-3.5 and GPT-4. We find that performance varied depending on several input choices, including table input format, content order, role prompting, and partition marks. Drawing from the insights gained through the benchmark evaluations, we propose *self-augmentation* for effective structural prompting, such as critical value / range identification using internal knowledge of LLMs. When combined with carefully chosen input choices, these structural prompting methods lead to promising improvements in LLM performance on a variety of tabular tasks, *e.g.*, TabFact(↑ 2.31%), HybridQA(↑ 2.13%), SQA(↑ 2.72%), Feverous(↑ 0.84%), and ToTTo(↑ 5.68%). We believe that our open source[1] benchmark and proposed prompting methods can serve as a simple yet generic selection for future research.

CCS CONCEPTS

• **Information systems** → **Information retrieval query processing**; • **Computing methodologies** → **Natural language processing**; **Natural language generation**.

*Yuan Sui and Mingjie Zhou made their contributions during their internships at Microsoft Research Asia, located in Beijing, China.
†Corresponding author.
[1]Please find code and data of the paper at https://github.com/microsoft/TableProvider.

WSDM '24, March 4–8, 2024, Mérida, Yucatán, Mexico.

ACM ISBN 979-8-4007-0371-3/24/03…$15.00
https://doi.org/10.1145/3616855.3635752

KEYWORDS

large language models, semi-structured data, structural understanding capabilities, benchmark

ACM Reference Format:
Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2024. Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM '24), March 4–8, 2024, Mérida, Yucatán, Mexico.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3616855.3635752

1 INTRODUCTION

Structured data consists of plain text blocks organized by predefined structures to compress recurring information. It makes data more manageable and facilitates data analysis and processing by machines. **Table** is one of such structured data types with many applications such as Table-based Question Answering (TQA) [8, 21], Table-based Fact Verification (TFV) [7, 39], Table-to-Text [37] and Column Type & Relation Classification [11, 20]. The adoption of structured data has significantly contributed to the advancement of information retrieval and knowledge extraction in web mining and content analysis [15, 34].

Prompt engineering has been proven as a highly effective method for in-context learning (ICL). Recent studies, such as "chain of thoughts" (CoT) [38] and "self-consistency" [36] or hybrid approaches using both generation and retrieval methods [1, 42] have demonstrated that LLMs, *e.g.*, GPT-X [4, 28] and FlanT5 [9], can solve complex mathematical reasoning tasks in both zero-shot and few-shot settings. Furthermore, Chen [6] illustrates that by using CoT with LLMs, GPT-3.5 shows impressive performance with just one-shot demonstration on several tabular tasks. These findings have opened new possibilities for the use of LLMs in structured data.

However, previous work has not provided comprehensive studies that examined whether LLMs can truly understand tabular data or given a detailed discussion of the extent to which LLMs have already achieved structural understanding capabilities. Furthermore, despite the remarkable success of LLMs in handling natural languages, their application to tabular data modality presents unique challenges: as different tables define structure and features in distinct ways and often lack straightforward transformation into sequential text (table serialization). Based on our survey, we believe that the process of table serialization, along with context and

corresponding queries, is highly flexible. That is, there is a lack of grounded consensus or comprehensive investigation on what constitutes a common-sense or exhaustive input design for LLMs on tabular tasks. Previous work used various input designs in an ad-hoc manner [6, 14, 16, 18, 20, 25, 37]. For example, TaPEx [25] uses special tokens to indicate components like headers <HEAD> and rows <ROW>; TABBIE [20] serialize tables by both row-wise and column-wise; while TableGPT [16] use a template-based method to serialize attribute-value pairs in each table record, *e.g.*, changing "name: Elon Musk" to "name is Elon Musk." and concatenating all the sentences according to the order of the records. The complex landscape of varied input design further complicates the challenges faced by researchers and developers in this field. Therefore, in this paper, our aim is to address the question: *What input designs and choices are the most effective in enabling LLMs to understand tables?*

In this paper, our goal is to address the chaotic landscape of input designs and determine whether LLMs can truly comprehend tabular data. We also aim to discuss the extent to which LLMs have already achieved in terms of their structural understanding capabilities. To achieve this, we propose a benchmark called *SUC* (structural understanding capabilities) to compare various input designs and create specific tasks in §3 that focus on each structural understanding capability of LLMs. To assess the effectiveness of multiple input choices, we conduct a series of experiments using different prompt variants. These variants include input format, format explanation, role prompting, partition mark [13], and zero-shot / one-shot approaches. The SUC benchmark offers a comprehensive comparison of multiple input designs, evaluating different aspects of structural understanding capabilities over table(s) as illustrated in §5. We then provide pragmatic guidance on how to better utilize LLM in understanding structured data in §4. Specifically, we propose a *model-agnostic* method called *self-augmented prompting* to directly boost the performance of LLM in downstream tabular-based tasks. This method motivates LLMs to generate intermediate structural knowledge by internally retrieving their own knowledge, *e.g.*, motivates LLMs to generate critical value / range identification by itself. These choices diverge from previous approaches like CoT and Zero-shot-CoT [22] by focusing on identifying effective methods for unlocking LLMs' capabilities to correctly comprehend structured information. We find that when combined with carefully chosen input choices, these structural prompting methods lead to promising improvements in LLM performance on various tabular reasoning tasks, *e.g.*, TabFact(↑ 2.31%), HybridQA(↑ 2.13%), SQA(↑ 2.72%), Feverous(↑ 0.84%), and ToTTo(↑ 5.68%) compared to baseline methods. See the results in §5.

Our exploration leads us to believe that 1) LLMs have basic structural understanding capabilities but are far from perfect, even on trivial tasks, *e.g.*, table size detection (detect the number of columns and rows in a table); 2) Choosing the right combination of input designs can significantly enhance LLMs' understanding of structured data. Different combinations of serialization functions and input options demonstrate noticeable performance gaps in downstream tasks (see §5). The disparity remains even when using GPT-4, validating the effectiveness of our benchmarking approach; 3) Self-augmented prompting is a simple model-agnostic method for better utilizing LLMs' internal knowledge and unraveling new possibilities to improve their structural understanding capabilities.

In summary, we propose using markup language like **HTML** with certain structural features like format explanation and partition mark, combined with self-augmented prompting, to fully leverage LLMs' internal knowledge and achieve better results in tabular reasoning tasks. **Our main contributions are:**

- We propose the SUC benchmark to evaluate the multiple structural understanding capabilities of LLMs.
- Through comprehensive experiments on the benchmark, we provide insights and guidelines on tabular input choices for future work (see §5).
- We propose self-augmentation as a method to enhance the performance of LLMs by leveraging internal knowledge. We verify the effectiveness of this simple but generic method on five tabular reasoning datasets.

2 PRELIMINARIES

2.1 Table Structure

Tabular data exhibit remarkable flexibility in diverse structures, as illustrated in [41]. These structures include relational tables, entity tables, matrix tables, layout tables, and more. Tables can have horizontal or vertical orientations and span the spectrum from flat to hierarchical. In this paper, we mainly focus on flat relational tables but also have some discussion on hierarchical tables, such as ToTTo [29]. In these tables, each row corresponds to a distinct record, while columns represent specific fields, without any hierarchical arrangement.

Tabular data also exhibit various approaches for formatting values, including text, numbers, date/time, formulas, and other relevant information. In particular, text plays a pivotal role in tables, capturing meta-information such as headers, notes, captions, and cells within the data region. On the other hand, numbers often involve arithmetic relationships like summation and proportion, as well as statistical attributes such as distribution and trends. Furthermore, tables commonly present meticulously organized numerical data, making it easy for reference and comparison. These structured numerical values are often documented using spreadsheet formulas [12]. The flexibility of tabular data poses unique challenges for LLMs, as different tables define structure and formatting in distinct ways. The gap between tabular data and natural languages (NL) hinders the application of NL reasoning to facilitate table reasoning.

2.2 Table Serialization & Splitting

Table serialization refers to the process of converting data from tables into a linear, sequential text format. This adaptation is essential for training and utilizing LLMs, especially for tasks like masked language modeling, where understanding and predicting language patterns is crucial. A simple serialization function is to serialize tables row-by-row. Many works such as TaPas [18], MATE [14], TableFormer [26], TUTA [37], and TURL [11] use this method. TaPEx [25] uses special tokens to indicate components like headers <HEAD> and rows <ROW>. TABBIE [20] serialize tables both by row-wise and column-wise. While TableGPT [16] use a template-based method to serialize attribute-value pairs in each table record.

Furthermore, most LLMs are inefficient in dealing with long sentences due to the quadratic complexity of self-attention [33, 35][2]. However, structured data typically contains dozens of components, which presents a significant challenge in terms of *memory* and *computational efficiency*. While Herzig et al. [18], Liu et al. [25] use naive methods to truncate the input based on a maximum sequence length, this approach may result in the loss of critical information and disrupt the structure of the entire table. In our experiments, we have predefined certain constraints to meet the LLM call request. For example, (1) to avoid potential disruption caused by truncation, we employ a random row sampling strategy when the number of tokens in the table exceeds a certain threshold, and (2) we append a 1-shot example based on the estimated remaining token capacity. Several meticulously crafted sequence serialization functions have been proposed as common practices in table serialization, including Dou et al. [13], Herzig et al. [18], Liu et al. [25], Shao et al. [31], Wang et al. [37], Xie et al. [39]. In this paper, we gather various commonly used serialization methods as baselines and conduct a fair comparison in Sec §3.

3 SUC BENCHMARK

In this section, we aim to develop a benchmark for comparing different input designs and investigating the structural understanding capabilities of LLMs. Specifically, we explore the following aspects: 1) *What input designs and choices are most effective in enabling LLMs to understand tables?*; 2) *To what extent do LLMs already possess structural understanding capabilities for structured data?* Additionally, we analyze the intricate trade-off of multiple combinations of input designs. Find the benchmark collection and pre-processing details in Sec §3.3.

3.1 Structural Understanding Capabilities

We categorize the essential abilities to comprehend table structures from a human point of view into two distinct folds, as illustrated in Figure 1.

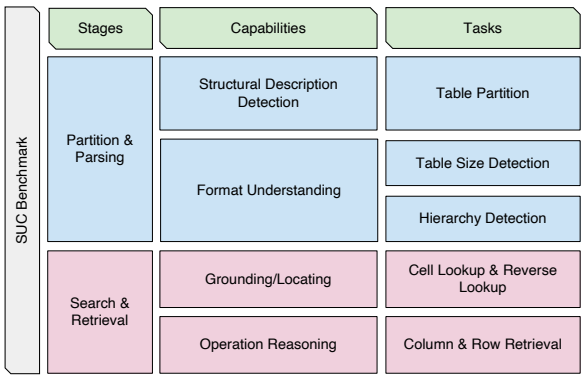

Figure 1: SUC Benchmark Overview

1) Partition & Parsing. Tabular datasets are always paired with knowledge from other sources to provide more context and solve a specific downstream task. For example, HybridQA [8] employs passage information, TabFact [7] and FEVEROUS [3] employs human annotation, and MultiModalQA [32] employs image information. However, the prerequisite for tackling these downstream tasks is the accurate partitioning of the data, which in turn requires the ability to distinguish tables from other supplementary information and an elementary understanding of the structural layout of tables.

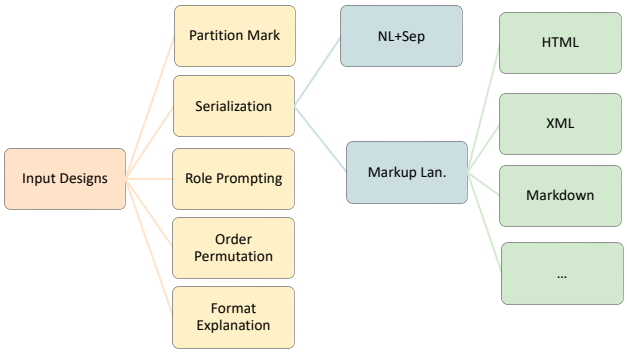

Figure 2: Input Designs for SUC Evaluation

Additionally, various table storage formats, including CSV, JSON, XML, markdown, HTML [2] and XLSX, have different levels of information compression and present different challenges for LLMs in understanding the table content. For example, a table stored in CSV format is organized in rows with column values separated by commas, while a table stored in XML format is represented as a nested set of tags. LLMs should first understand the format or layout of the table and then grasp its content. To our knowledge, no previous work has discussed the impact of these various storage formats. We aim to determine whether LLMs have the ability to correctly parse different formatting sources and identify which type of input design is most suitable for LLMs. It is also possible that LLMs already have the capability to handle all types of storage formats. The specific input designs can be found in Figure 2.

2) Search & Retrieval. In addition to the capabilities mentioned above, the ability to accurately search and retrieve information from specific positions within structured data is crucial for LLMs. This capability is highly relevant to a wide range of downstream tasks, including but not limited to Table-QA and Column Type & Relation Classification. It empowers LLMs to effectively identify and extract relevant information from structured data based on user queries or requests. For instance, consider a user asking, "For the Olympic events that took place after 2014, which event had an older flag bearer?" In order to answer this query, the LLM needs to first locate all the Olympic events that satisfy the time criterion, then compare the ages of the flag bearers associated with each event, and finally determine and return the event with the oldest flag bearer. The process of locating the relevant information within the structured data is achieved through careful analysis of the data's structure and the identification of the target cell or cells. By disentangling the search & retrieval capabilities from the downstream tasks of LLMs, we gain valuable insights into the inner learning process of LLMs when it comes to tabular data.

[2]The maximum sequence length of text-Davinci-003 is constrained to 4k tokens.

3.2 Task Design

We have designed several specific tasks to assess the capabilities of LLMs in understanding tables (See concrete prompt design in Table 1). These tasks are designed in increasing difficulty.

Table Partition. This task assesses the capability of LLMs to identify the structure of tables. The LLM is required to detect the boundaries of the tables within a given user input design. This input design may include various types of supplementary information, such as "descriptions", "context", "statements" and "user queries". Formally, given an input design, $D = d_1, d_2, ...$, where each part d_i is a "versatile" sequence containing supplementary information such as description, context, statement, or user queries. For easy evaluation and comparison, we constrain LLMs to output a tuple of table boundary with the head token b_h and the end token b_e that includes the table content, as $B = (b_h, b_e)$.

Table Size Detection. This task is essential to reveal the LLM's capability to correctly parse structural information. The size of a table is an important feature that is often overlooked. In fact, the table size feature represents direct constraints to how many rows and columns are encoded in a table. For instance, if a table only has three columns, the output should not consider answers outside this scope. Formally, given a table with m rows and n columns, a correct answer from a LLM should be (m, n).

Merged Cell Detection. This task assesses LLM's capability to parse structural information by detecting merged cells. Merged cells are special structures in table construction where two or more adjacent cells are combined to create a larger cell. To test the robustness of LLMs, we consider merged cells as a feature in hierarchical spreadsheet tables. Formally, given a table with some merged cells, the LLM is required to detect the merged cell index (r_i, c_j). Note that any index in the merged cell that matches the condition will be considered correct.

Cell Lookup & Reverse Lookup. This task reveals the capability to search and retrieve structural information. The LLM is required to accurately search and retrieve the cell value from a specific position. This task relies on the capabilities of information partitioning and parsing. In this task, if multiple cells with the same value are found, the LLM should retrieve their positions $(p_i, p_j), \cdots, (p_i^n, p_j^n)$. Conversely, given a specific cell position (p_i, p_j), the LLM should retrieve the corresponding cell value c_i.

Column & Row Retrieval. This task assesses the LLM's capability to search and retrieve structural information by listing cell values. For column retrieval, the LLM is required to list the cell values c_j, c_j^n of a specific column name C_i from the given table. Similarly, for row retrieval, the LLM should list the cell values of a specific row index. In the evaluation process, we consider the prediction to be correct if the predicted value list matches the ground truth value list. We expect the performance of column/row retrieval tasks to be better than that of cell lookup and reverse lookup tasks, as using column/row indices to locate specific value lists is more common.

3.3 Data Collection and Reformatting of SUC

We collect structured data from various public datasets, *e.g.*, TabFact [7], FEVEROUS [3], SQA [21], HybridQA [8] and ToTTo [29].

Table 1: Input design of each task in our benchmark

Task	Input
Table Partition	What is the first token (cell value instead of separator |) of the given table? What is the end token (cell value instead of separator |) of the given table? Answer questions one by one and use | to split the answer.
Cell Lookup	What is the position of the cell value cell_value? Use row index and column index to answer
Reverse Lookup	What is the cell value of row index, column index ? Only output the cell value without other information
Column Retrieval	What is the column name with the index column_idx of the following table? Only give the column name without any explanation
Row Retrieval	What are the cell values of the row_idx row in following table? Only list the cell values one by one using | to split the answers
Size Detection	How many rows in the table? How many columns in the table. Answer the questions one by one and use | to split the answer
Merged Cell Detection	What is the column index of the cell which span is over 1. use | to split the answer (e.g., 3 | 4), the column index starts from 0. If there's no answer, return None

All the tables are from Wikipedia. We only consider the structural portions of the original datasets, which are labeled with "table," "rows," or "headers," and exclude the other parts like "ID," "Answer," "Question," "FileName,". To identify a specific value within the structured data, we append each parsed sample with a unique question. Most of these questions are one sentence long, with a median length of 15 words. For example, "How many rows (columns) are in the table?" Each question is accompanied by a set of reference answers ("groundtruth") sourced from the original datasets. For better evaluation, most of these questions are paired with some constraints such as "Answer the questions one by one and use "|" to split the answer.". We evaluate these questions using GPT-3.5 (Text-Davinci-003)[3] and manually eliminate any question that the model consistently answers correctly when multiple random samples are generated at a nonzero temperature[4]. For the merged cell detection task, we only sample from ToTTo dataset since this is the only source paired with the merged cell. For each task setting, we randomly sample 1,500 tables for testing with a guaranteed table distribution.

One-shot Setting. The SUC benchmark is designed as a one-shot in-context learning benchmark for tabular tasks. This means that the model can access one example from the SUC and may gain some context when generating the answers. Large Language Models (LLMs) have shown impressive capability in following few-shot prompts to accomplish unseen tasks without any fine-tuning [23]. This emergent capability is not captured by small language models. SUC leverages this property to better reveal the potential capabilities that LLMs may lack. We also conduct experiments using the zero-shot setting for comparison (See Table 3).

[3]We perform the experiments through the public playground of OpenAI GPT-3.5 in https://beta.openai.com/playground/.

[4]Temperature controls the randomness of the generation process. As the temperature approaches zero, the model becomes more deterministic and repetitive with very limited variation. Here, we set the temperature to 0.7 when creating the question and set the temperature to 0 when performing other experiments

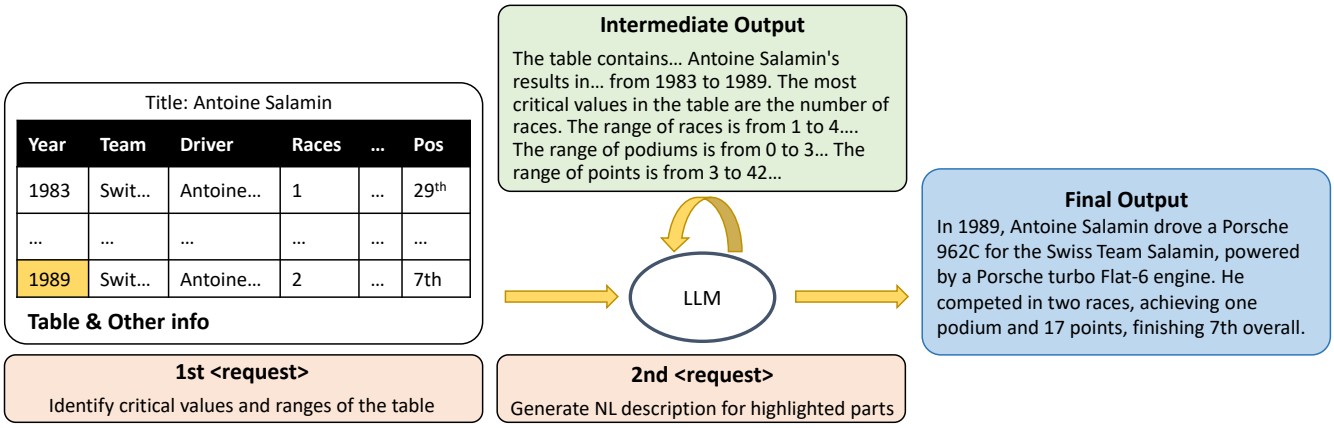

Figure 3: Illustration of self-augmented prompting. This process consists of two phases: 1) using self-augmented prompts to ask the LLM to generate additional knowledge (intermediate output) about the table; 2) incorporating the self-augmented response into the second prompt to request the final answer for a downstream task. As depicted in the figure, the LLM is able to identify important values in the table, which assists in generating a more accurate answer for the downstream task.

3.4 Evaluation

We evaluate the benchmark using common input designs for table reasoning tasks and apply the methods to different LLMs for a deeper analysis. Specifically, we consider CSV, JSON, XML, markdown, HTML [2], and XLSX as different format options. Each format represents a different level of information compression and poses different challenges for LLMs to understand the table content. we also consider using the most common way of concatenating a special token [39, 40] as a separator, such as "|", as the baseline. The comparison numbers can be found in Table 2. We also explore other input design options, such as grammar explanation, partition mark [13], role prompting [25], and format explanation, as augmentations for input designs. More details can be found in Figure 2 and Table 3. In particular, we consider using the *accuracy* for each task's evaluation. To ensure better evaluation, we have added some constraints to the output format. For example, in the table partition task, we include the instruction "Answer questions one by one and use '|' to split the answer." Based on empirical observations, over 90% of the answers follow these specific format instructions. For the remaining 10% of samples, we apply a semantic-parsing strategy using regular expressions (Re) [5] to parse the answers.

4 STRUCTURAL PROMPTING

Our findings and insights over the SUC comparisons (See Sec §5) have led us to the discovery that 1) LLMs have the basic structural understanding capabilities but are far from perfect, even on some trivial tasks, *e.g.*, table size detection; 2) Correctly choosing the combination of input designs is a potential factor in boosting LLMs understanding over tabular data. In this section, we propose a simple and generic method, **self-augmented prompting**, to generate additional constraints using LLMs' self-knowledge. We find that when combined with carefully chosen input choices, these structural prompting methods lead to promising improvements on a variety of tabular downstream tasks (See Table 4).

Recently, CoT [38] has been discovered to empower LLMs to perform complex reasoning over text and lead to a long line of work [10, 22, 24, 36]. By providing the model with several exemplars of reasoning chains, LLMs can learn to follow the template to solve difficult unseen tasks. Inspired by these works, we propose a simple, generic, and effective method, *self-augmented prompting*, to generate intermediate structural knowledge based on the internal retrieving of LLMs' self knowledge base. We design several ways to squeeze knowledge from LLM (see Table 4). For example, we ask LLM to generate the format specification, which intends to clarify the input format pattern by LLM itself. These choices diverge from previous approaches like CoT and Zero-shot-CoT [22] by focusing on identifying effective methods for unlocking LLMs' capabilities to correctly comprehend structured information. Additionally, this method is *model-agnostic*, that is, any standard structural data reasoning tasks can be used as the backbone, and can also be integrated with other prompting-based methods like self-consistency [36].

Formally, self-augmented prompting is a simple idea that utilizes prompting twice to leverage the capabilities of LLMs in understanding structured data, as shown in Figure 3. In the first prompt, the original task of "requesting" information is replaced with a simple demand to "Identify critical values and ranges of the last table related to the statement". Each demand represents an important aspect of structural information. The purpose of this replacement is to unlock the reasoning abilities of LLMs for complex reasoning over structured data. The prompted text is then fed into the LLM model, which generates a subsequent sentence containing specific structural information. In the second prompt, the generated subsequent sequence is appended to the task request and fed into the LLM model to generate the final answer. Refer to Table 4 for a comparison of the experiment results using self-augmented prompting.

[5] https://docs.python.org/3/library/re.html

Table 2: Micro results of the benchmark. Change order [39] refers to put external text (like questions, statement) ahead of tables. Noted that "GPT-4" refers to the evaluation outcomes utilizing the GPT-4 model. Given the resource-intensive nature of GPT-4 calls, we only conducting the GPT-4 inference test on a subset of 300 samples (randomly sampled) from each task set. Each column follows the roles of graded color scale, *i.e.*, the deeper color refers to better perf.

Format	Table Partition		Cell Lookup		Reverse Lookup		Column Retrieval		Row Retrieval		Size Detection		Merged Cell Detection	
	Acc	GPT-4	Acc	GPT-4	Acc	GPT-4	Acc	GPT-4	Acc	GPT-4	Acc	GPT-4	Acc	GPT-4
NL + Sep	93.00%	96.78%	39.67%	72.48%	52.00%	59.12%	60.67%	66.32%	31.00%	48.67%	42.00%	73.12%	71.33%	74.98%
Markdown	92.33%	**98.32%**	43.33%	71.93%	51.00%	57.32%	35.33%	60.12%	**42.33%**	49.98%	40.67%	82.12%	**78.00%**	**82.64%**
JSON	94.00%	97.12%	42.67%	68.32%	54.33%	58.12%	54.33%	64.32%	29.00%	48.32%	42.67%	76.43%	73.33%	78.98%
XML	96.00%	97.64%	43.33%	72.28%	**55.00%**	**60.32%**	41.33%	68.28%	41.00%	**50.28%**	43.67%	80.21%	75.00%	80.32%
HTML	**96.67%**	**98.32%**	**44.00%**	**73.34%**	47.33%	59.45%	**63.33%**	**69.32%**	42.00%	50.19%	**67.00%**	**83.43%**	76.67%	81.28%

Table 3: Micro ablation results of the input designs over benchmark.

Input Design	Table Partition		Cell Lookup		Reverse Lookup		Column Retrieval		Row Retrieval		Size Detection		Merged Cell Detection	
	Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ
Markup Lan. HTML	**96.67%**	0.00%	44.00%	0.00%	47.33%	0.00%	63.33%	0.00%	42.00%	0.00%	67.00%	0.00%	76.67%	0.00%
w/o format explanation	92.00%	-4.67%	52.00%	8.00%	52.33%	5.00%	64.33%	1.00%	36.00%	-6.00%	**78.00%**	11.00%	**77.67%**	1.00%
w/o partition mark	**98.00%**	1.33%	**59.00%**	15.00%	53.00%	5.67%	66.00%	2.67%	39.67%	-2.33%	**72.00%**	5.00%	70.33%	-6.33%
w/o role prompting	95.00%	3.00%	40.67%	-11.33%	44.67%	-7.67%	59.00%	-5.33%	39.33%	3.33%	**69.00%**	-9.00%	76.00%	-1.67%
w/o change order	**96.67%**	0.00%	52.33%	8.33%	40.67%	-6.67%	55.67%	-7.67%	31.67%	-10.33%	52.67%	-14.33%	65.67%	-11.00%
w/o 1-shot	63.00%	-33.67%	9.33%	-34.67%	17.33%	-30.00%	50.00%	-13.33%	30.00%	-12.00%	16.67%	-50.33%	38.00%	-38.67%
GPT-4 w/ Lan. HTML	**98.32%**	1.65%	73.34%	29.34%	59.45%	12.12%	69.32%	5.99%	50.19%	8.19%	83.43%	16.43%	81.28%	4.61%

Based on the empirical observations, it is evident that structural information plays a crucial role in comprehending a table. Researchers such as Aghajanyan et al. [2], Xie et al. [39], Yin et al. [40] have made progress by incorporating prompts with special tokens to encode different structural information. Building on their work and the findings from the SUC benchmark results, we explore the concept of manual prompt engineering as an additional technique for self-augmented prompting. Specifically, we consider extracting structural information from the raw input and incorporating it into the input itself. This can involve using cell addresses and clearly indicating the number of rows and columns in the table. Such augmentation aims to provide additional knowledge and constraints, thereby improving the LLM's ability to reason in tabular downstream tasks. We have observed that the LLM performs poorly in the task of table size detection (refer to Section 3), which motivates us to include structural features in the input. For example, we append information about the table size and merged cell positions to create a more structure-aware in-context learning environment for downstream tasks. Our ablation study in Sec §5.2.1 shows that appending table size and merged cell position leads to an improvement in the LLM's performance on downstream tasks.

5 EXPERIMENTS

5.1 Experiment Settings

Models. In this study, we evaluate the performance on GPT-3.5 [28] and GPT-4 [27]. Unless otherwise specified, we utilize text-davinci-003 in all experiments. Specifically, we set the hyper-parameter temperature to 0, top_p to 1, with n set to 1 when performing the experiments; **Downstream Tasks and Datasets.** In addition to evaluate LLMs' capabilities toward understanding structured data through our benchmark. We also conduct experiments on five typical tabular downstream tasks. The datasets are shown as follows, and the evaluation number can be found in Table 4.

Specifically, we use **(1) SQA** which is composed of 6,066 question sequences (2.9 questions per sequence on average), constructed by decomposing a subset of highly compositional WTQ questions; **(2) HybridQA** which requires reasoning on heterogeneous information rather than homogeneous information alone, which involves 62,682 questions. Each question is aligned with a Wikipedia table and multiple free-form corpora linked with the entities in the table. The questions are designed to aggregate both tabular information and text information, *i.e.*, lack of either form would render the question unanswerable; **(3) ToTTo** which is a high-quality English table-to-text dataset with more than 100,000 examples in which a table from Wikipedia with highlighted cells is paired with a sentence that describes the highlighted cells. The task is like given a Wikipedia table with row names, column names and table cells, with a subset of cells highlighted, generate a natural language description for the highlighted part of the table; **(4) FEVEROUS** which is a fact verification dataset consisting of 87,026 verified claims. Each claim is annotated with evidence in the form of sentences and/or cells from tables in Wikipedia, as well as a label indicating whether this evidence supports, refutes, or does not provide enough information to reach a verdict; **(5) TabFact** which is a fact verification dataset in which the tables were extracted from Wikipedia and sentences were written by crowd workers.

5.2 Results

5.2.1 Benchmark Highlights. Comprehensive evaluations of different structural understanding tasks with various input designs

Table 4: Downstream tasks evaluation. SA. refers to Self-augmented Prompting.

Type	Choice	TabFact	HybridQA	SQA	Feverous	ToTTo			
		Acc	Acc	Acc	Acc	BLEU-1	BLEU-2	BLEU-3	BLEU-4
1-shot	1-shot	72.04%	46.07%	73.81%	75.56%	72.43%	44.36%	27.01%	17.24%
1-shot	w/o table size	71.33%	45.52%	72.91%	74.66%	72.30%	44.23%	27.14%	17.25%
1-shot	w/o partition mark	71.25%	45.48%	73.09%	75.11%	71.18%	43.17%	26.36%	16.34%
1-shot	w/o format explanation	70.87%	45.39%	71.69%	75.97%	70.54%	43.59%	26.52%	16.74%
1-shot	w/o role prompting	71.35%	46.05%	73.39%	75.52%	70.61%	43.10%	26.02%	16.15%
SA	self format explanation	72.23%	46.12%	73.91%	76.15%	74.18%	45.25%	27.32%	18.34%
SA	self critical values and ranges identification	74.35%	48.20%	76.53%	76.32%	80.83%	47.96%	30.68%	22.92%
SA	self structural information description	73.42%	46.97%	75.97%	77.28%	78.93%	46.91%	28.94%	19.32%

over SUC are presented in Table 3. The results show that the system's overall accuracy gets highest when using the HTML markup language with format explanations and role prompts, and without order change, achieving a 65.43% overall accuracy on seven tasks. It indicates that the LLM has significant potential for understanding the structural information of tables in this specific format. However, it is also evident that the LLM's performance is negatively impacted when certain features are removed, especially when the prompt example is removed. We give some highlights associated with the benchmark results as follows:

NL+Sep vs. Markup Lan. We compare the use of natural language with specific separators (NL+Sep) and markup languages such as HTML, XML, and JSON. Even "NL+Sep" is commonly used in tabular downstream tasks [6, 18, 25, 40], however, our results show that using markup languages, specifically HTML, outperforms "NL+Sep" with a 6.76% improvement. We assume that the training process of the LLMs involves code tuning and that the training dataset contains a significant amount of web data. As a result, the LLM is more familiar with HTML and XML formats when interpreting tables. (For more information about the GPT-3.5 training, see [28]).

Table 5: Main results of the downstream tasks ablation study

Format	TabFact	HybridQA	SQA	Feverous	ToTTo
	Acc	Acc	Acc	Acc	BLEU-4
NL + Sep	70.26%	45.02%	70.41%	75.15%	**12.70%**
Markdown	68.40%	45.88%	66.59%	71.88%	8.57%
JSON	68.04%	42.40%	70.39%	73.84%	8.82%
XML	70.00%	47.20%	70.74%	73.14%	8.82%
HTML	**71.33%**	**47.29%**	**71.31%**	**75.20%**	12.30%
GPT-4 w/ HTML	78.40%	56.68%	75.35%	83.21%	20.12%

1-shot vs. 0-shot. A notable finding is that the system's performance drops significantly when it is in a zero-shot setting, with an overall accuracy decrease of 30.38% on all tasks using HTML format. This indicates that learning structural information is highly dependent on in-context learning. This is particularly significant for tasks such as size detection and merged cell detection, which are closely related to the ability to parse structural information.

External information should appear ahead of tables. In order to understand the impact of the order on the input design, we observed that when we manually placed external information such as questions and statements behind the table, there was an overall 6.81% decrease in performance across all tasks. One possible explanation for this is that placing external information ahead of tables could assist LLM in better generalization and gaining more context regarding the structural information of tables.

Partition mark and format explanation may undermine Search and Retrieval capability. Partition mark [13] is commonly used in input designs. Inspired by the partition mark, we propose another similar choice called "format explanation". It provides an additional explanation of the adopted format. For example, in the case of HTML format, we explain that "Each table cell is defined by a <td> and a </td> tag; Each table row starts with a <tr> and ends with a </tr> tag; th stands for table header." However, when it comes to the task of Cell Lookup, adding partition marks and format explanations actually results in a decrease in performance across all input designs. This suggests that such additional structural information may bias the searching and retrieval process of LLM over the tabular structure. However, adding partition marks or format explanations does show some benefits for specific tasks such as merged cell detection. To provide a clearer understanding of the impact of adding additional explanations or special tokens, we conducted experiments as shown in Table 4. The results reveal that while they may undermine the search and retrieval capability of LLMs, they still improve overall performance in downstream tasks.

The SUC benchmark provides a comprehensive comparison using multiple input designs to evaluate the structural understanding capabilities of tables. Based on the findings, guidelines are proposed to address the questions mentioned in Sec §3:

- 1) LLMs have basic structural understanding capabilities, but are far from perfect, even for some trivial tasks, *e.g.*, table size detection;
- 2) Choosing the right combination of input designs is a potential factor in improving the performance of LLMs when working with structured data.

5.2.2 Downstream Tasks. Table 4 gives the comparison using self-augmented prompting with the trick of structural feature detection. We find one notable insight that the models perform better using self-augmented prompting than 1-shot, as seen in the "SA" rows

where the models are given self-generated information: format explanation, key range and values identification, and structural information description. These rows have higher accuracy and BLEU scores than the "1-shot" rows. Especially for generating key range and values identification gives an overall 3.26% performance improvement than 1-shot setting on all the tasks. Furthermore, manually prompting structural features like "table size" from the input has a slight impact on the model's performance on downstream tasks. The drop of table size option gives an overall 0.4% decrease, as seen by the small difference between the "1-shot" row and the "1-shot w/o table size" row.

Table 6: Format explanation example from FEVEROUS

Manual Prompt	Self-augmented Prompt Generation
Each table cell is defined by a <td> and a </td> tag. Each table row starts with a <tr> and ends with a </tr> tag. th stands for table header.	'15 rows and 4 columns, with the first column being an index, the second column being empty, the third column being the name of the Marshal, the fourth column being the date of promotion, and the fifth column being the defence branch.\nThe table is defined by HTML tags, with each table cell being defined by a <td> and a </td> tag, and each table row starting with a <tr> and ending with a </tr> tag.\nThe table header is denoted by the th tag.'

Based on the comparison between the "1-shot w/o format explanation" row and the "SA self format explanation" row, we observe that manually labeling the format explanation may have a negative impact on downstream tasks like FEVEROUS. This is because the table structure in FEVEROUS is more irregular, with numerous segments and subtables. These structural complexities pose significant challenges for GPT-3.5. Additionally, manually-crafted knowledge is more general and cannot cover detailed information of this nature. On the other hand, self-augmented prompting can learn patterns independently and generate more comprehensive and helpful cues to address the questions. We provide an example of a format explanation from FEVEROUS using two prompt designs in Table 6.

6 RELATED WORK

In-context Learning with LLMs. Large language models, such as GPT-3 [4], Instruct-GPT, and Codex [5], have demonstrated their capability as few-shot reasoners in natural language-related tasks. The effectiveness of this capability is influenced by factors such as the model size, the amount of data used, and the available computing power. Recent studies [9, 10, 30] have proposed various methods for training these large language models (LLMs). These models have exhibited an impressive ability to perform tasks that they haven't been specifically fine-tuned for, which is an emergent capability not observed in smaller language models.

Intermediate of Prompt Engineering. Recently, several intermediate prompt engineering methods have been proposed following "CoT" [38]. CoT provides a few examples with explanations of the reasoning process. This step-by-step reasoning approach helps LLMs generate more accurate results. However, according to [38], "CoT only yields performance gains when used with models of nearly 100B parameters." Smaller models tend to produce illogical chains of thought, resulting in lower accuracy compared to standard prompting. Typically, the performance boost from CoT prompting is proportional to the model's size. Zero-shot chain of Thought (Zero-shot-CoT) prompting is a follow-up to CoT that introduces an incredibly simple zero-shot prompt. By appending the words "Let's think step by step." to the end of a question, LLMs can generate a chain of thoughts that answers the question. Extracting answers from this chain of thought leads to more accurate results. Another follow-up to CoT is Self-consistency [36], which generates multiple chains of thoughts and selects the majority answer based on a voting strategy as the final answer. Self-consistency has shown improvements in arithmetic, commonsense, and symbolic reasoning tasks. Even when regular CoT is found to be ineffective, self-consistency can still enhance results. In addition, Liu et al. [24] propose the generated knowledge approach, which prompts the LLM to generate potentially useful information about the question before generating a response.

7 CONCLUSION

In this paper, we propose a benchmark to compare various input designs in order to study the structural understanding capabilities of LLMs on tables. Surprisingly, we obtain some insights of the input designs and the comparison reveal that LLMs have the basic capabilities towards understanding structural information of tables. We also give some guidance on how to apply our benchmark insights on downstream tasks and propose a simple, generic but effective method, *i.e.*, self-augmented prompting, by generating additional knowledge with LLMs self-knowledge. We believe this study will be beneficial for table-based, even structured data based research, or serve as a auxiliary tool to help better understand the table(s) from structural perspectives.

ETHICAL CONSIDERATIONS

Structured data often includes metadata, which provides additional information about the data and helps to provide context (*e.g.*, column names, data types, *etc.*). Interpreting and utilizing metadata is a challenge when the meaning and significance of the structured data may not be immediately apparent and must be inferred from the metadata and other contextual clues. This capability is highly dependent on downstream tasks, such as column type prediction [19] and dimension/measure classification [17]. We believe that understanding this challenge is an important area of research. However, due to space limitations, we will leave this section for further exploration. Furthermore, our method is primarily designed for languages with limited morphology, such as English. The scalability of our approach to longer texts is a topic that we will explore in more detail.

REFERENCES

[1] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning with LLMs. https://doi.org/10.48550/arXiv.2305.11860 arXiv:2305.11860 [cs]

[2] Armen Aghajanyan, Dmytro Okhonko, Mike Lewis, Mandar Joshi, Hu Xu, Gargi Ghosh, and Luke Zettlemoyer. 2021. HTLM: Hyper-Text Pre-Training and Prompting of Language Models. arXiv:2107.06955 [cs] http://arxiv.org/abs/2107.06955

[3] Rami Aly, Zhijiang Guo, Michael Schlichtkrull, James Thorne, Andreas Vlachos, Christos Christodoulopoulos, Oana Cocarascu, and Arpit Mittal. 2021. FEVEROUS: Fact Extraction and VERification Over Unstructured and Structured Information. https://doi.org/10.48550/arXiv.2106.05707 arXiv:2106.05707 [cs]

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Dhariwal, et al. 2020. Language Models Are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. https://doi.org/10.48550/arXiv.2107.03374 arXiv:2107.03374 [cs]

[6] Wenhu Chen. 2022. Large Language Models Are Few(1)-Shot Table Reasoners. https://doi.org/10.48550/arXiv.2210.06710 arXiv:2210.06710 [cs]

[7] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2020. TabFact: A Large-Scale Dataset for Table-Based Fact Verification. https://doi.org/10.48550/arXiv.1909.02164 arXiv:1909.02164 [cs]

[8] Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 1026–1036. https://doi.org/10.18653/v1/2020.findings-emnlp.91

[9] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022. Scaling Instruction-Finetuned Language Models. https://doi.org/10.48550/arXiv.2210.11416 arXiv:2210.11416 [cs]

[10] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. https://doi.org/10.48550/arXiv.2110.14168 arXiv:2110.14168 [cs]

[11] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2020. TURL: Table Understanding through Representation Learning. https://doi.org/10.48550/arXiv.2006.14806 arXiv:2006.14806 [cs]

[12] Haoyu Dong, Zhoujun Cheng, Xinyi He, Mengyu Zhou, Anda Zhou, Fan Zhou, Ao Liu, Shi Han, and Dongmei Zhang. 2022. Table Pre-training: A Survey on Model Architectures, Pre-training Objectives, and Downstream Tasks. https://doi.org/10.48550/arXiv.2201.09745 arXiv:2201.09745 [cs]

[13] Longxu Dou, Yan Gao, Mingyang Pan, Dingzirui Wang, Wanxiang Che, Dechen Zhan, and Jian-Guang Lou. 2022. UniSAr: A Unified Structure-Aware Autoregressive Language Model for Text-to-SQL. arXiv:2203.07781 [cs] http://arxiv.org/abs/2203.07781

[14] Julian Martin Eisenschlos, Maharshi Gor, Thomas Müller, and William W. Cohen. 2021. MATE: Multi-View Attention for Table Transformer Efficiency. https://doi.org/10.48550/arXiv.2109.04312 arXiv:2109.04312 [cs]

[15] Björn Engelmann, Timo Breuer, and Philipp Schaer. 2023. Simulating Users in Interactive Web Table Retrieval. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management* (Birmingham, United Kingdom) *(CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 3875–3879. https://doi.org/10.1145/3583780.3615187

[16] Heng Gong, Yawei Sun, Xiaocheng Feng, Bing Qin, Wei Bi, Xiaojiang Liu, and Ting Liu. 2020. TableGPT: Few-Shot Table-to-Text Generation with Table Structure Reconstruction and Content Matching. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 1978–1988. https://doi.org/10.18653/v1/2020.coling-main.179

[17] Xinyi He, Mengyu Zhou, Jialiang Xu, Xiao Lv, Tianle Li, Yijia Shao, Shi Han, Zejian Yuan, and Dongmei Zhang. 2022. Inferring Tabular Analysis Metadata by Infusing Distribution and Knowledge Information. https://doi.org/10.48550/arXiv.2209.00946 arXiv:2209.00946 [cs]

[18] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pre-Training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 4320–4333. https://doi.org/10.18653/v1/2020.acl-main.398

[19] Madelon Hulsebos, Çağatay Demiralp, and Paul Groth. 2022. GitTables: A Large-Scale Corpus of Relational Tables. https://doi.org/10.48550/arXiv.2106.07258 arXiv:2106.07258 [cs]

[20] Hiroshi Iida, Dung Thai, Varun Manjunatha, and Mohit Iyyer. 2021. TABBIE: Pretrained Representations of Tabular Data. https://doi.org/10.48550/arXiv.2105.02584 arXiv:2105.02584 [cs]

[21] Mohit Iyyer, Wen-tau Yih, and Ming-Wei Chang. 2017. Search-Based Neural Structured Learning for Sequential Question Answering. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vancouver, Canada, 1821–1831. https://doi.org/10.18653/v1/P17-1167

[22] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models Are Zero-Shot Reasoners. https://doi.org/10.48550/arXiv.2205.11916 arXiv:2205.11916 [cs]

[23] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. arXiv:2109.07958 [cs] http://arxiv.org/abs/2109.07958

[24] Jiacheng Liu, Alisa Liu, Ximing Lu, Sean Welleck, Peter West, Ronan Le Bras, Yejin Choi, and Hannaneh Hajishirzi. 2022. Generated Knowledge Prompting for Commonsense Reasoning. https://doi.org/10.48550/arXiv.2110.08387 arXiv:2110.08387 [cs]

[25] Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. 2022. TAPEX: Table Pre-Training via Learning a Neural SQL Executor. https://doi.org/10.48550/arXiv.2107.07653 arXiv:2107.07653 [cs]

[26] Ahmed Nassar, Nikolaos Livathinos, Maksym Lysak, and Peter Staar. 2022. TableFormer: Table Structure Understanding with Transformers. https://doi.org/10.48550/arXiv.2203.01017 arXiv:2203.01017 [cs]

[27] OpenAI. 2023. GPT-4 Technical Report. https://doi.org/10.48550/arXiv.2303.08774 arXiv:2303.08774 [cs]

[28] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training Language Models to Follow Instructions with Human Feedback. https://doi.org/10.48550/arXiv.2203.02155 arXiv:2203.02155 [cs]

[29] Ankur P. Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. 2020. ToTTo: A Controlled Table-To-Text Generation Dataset. https://doi.org/10.48550/arXiv.2004.14373 arXiv:2004.14373 [cs]

[30] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Song, et al. 2022. Scaling Language Models: Methods, Analysis & Insights from Training Gopher. https://doi.org/10.48550/arXiv.2112.11446 arXiv:2112.11446 [cs]

[31] Yijia Shao, Mengyu Zhou, Yifan Zhong, Tao Wu, Hongwei Han, Shi Han, Gideon Huang, and Dongmei Zhang. 2022. FormLM: Recommending Creation Ideas for Online Forms by Modelling Semantic and Structural Information. https://doi.org/10.48550/arXiv.2211.05284 arXiv:2211.05284 [cs]

[32] Alon Talmor, Ori Yoran, Amnon Catav, Dan Lahav, Yizhong Wang, Akari Asai, Gabriel Ilharco, Hannaneh Hajishirzi, and Jonathan Berant. 2021. MultiModalQA: Complex Question Answering over Text, Tables and Images. https://doi.org/10.48550/arXiv.2104.06039 arXiv:2104.06039 [cs]

[33] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2022. Efficient Transformers: A Survey. https://doi.org/10.48550/arXiv.2009.06732 arXiv:2009.06732 [cs]

[34] Mohamed Trabelsi, Zhiyu Chen, Shuo Zhang, Brian D. Davison, and Jeff Heflin. 2022. StruBERT: Structure-aware BERT for Table Search and Matching. In *Proceedings of the ACM Web Conference 2022*. ACM. https://doi.org/10.1145/3485447.3511972

[35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[36] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. https://doi.org/10.48550/arXiv.2203.11171 arXiv:2203.11171 [cs]

[37] Zhiruo Wang, Haoyu Dong, Ran Jia, Jia Li, Zhiyi Fu, Shi Han, and Dongmei Zhang. 2021. TUTA: Tree-Based Transformers for Generally Structured Table Pre-Training. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1780–1790. https://doi.org/10.1145/3447548.3467434 arXiv:2010.12537 [cs]

[38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. https://doi.org/10.48550/arXiv.2201.11903 arXiv:2201.11903 [cs]

[39] Tianbao Xie, Chen Henry Wu, Peng Shi, Ruiqi Zhong, Torsten Scholak, Michihiro Yasunaga, Chien-Sheng Wu, Ming Zhong, Pengcheng Yin, Sida I. Wang, Victor Zhong, Bailin Wang, Chengzu Li, Connor Boyle, Ansong Ni, Ziyu Yao,

Dragomir Radev, Caiming Xiong, Lingpeng Kong, Rui Zhang, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. 2022. UnifiedSKG: Unifying and Multi-Tasking Structured Knowledge Grounding with Text-to-Text Language Models. https://doi.org/10.48550/arXiv.2201.05966 arXiv:2201.05966 [cs]

[40] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. 2020. TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 8413–8426. https://doi.org/10.18653/v1/2020.acl-main.745

[41] Shuo Zhang and Krisztian Balog. 2020. Web Table Extraction, Retrieval and Augmentation: A Survey. https://doi.org/10.48550/ARXIV.2002.00207

[42] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. 2023. Planning with Large Language Models for Code Generation. https://doi.org/10.48550/arXiv.2303.05510 arXiv:2303.05510 [cs]